

GRL: Knowledge graph completion with GAN-based reinforcement learning

Qi Wang^{a,*}, Yuede Ji^b, Yongsheng Hao^c, Jie Cao^d

^a Fudan University, China

^b Department of Electrical and Computer Engineering, George Washington University, USA

^c Network Center, Nanjing University of Information Science and Technology, China

^d Xuzhou University of Technology, China

ARTICLE INFO

Article history:

Received 20 February 2020

Received in revised form 15 August 2020

Accepted 1 September 2020

Available online 23 September 2020

Keywords:

Knowledge graph

Knowledge graph completion

Reinforcement learning

Deep learning

ABSTRACT

Knowledge graph completion intends to infer the entities that need to be queried through the entities and relations known in the knowledge graphs. It is used in many applications, such as question and answer systems, and searching engines. As the completion process can be represented as a Markov process, existing works would solve this problem with reinforcement learning. However, there are three issues blocking them from achieving high accuracy, which are reward sparsity, missing specific domain rules, and ignoring the generation of knowledge graphs. In this paper, we design a generative adversarial net (GAN)-based reinforcement learning model, named GRL, for knowledge graph completion. First, GRL employs the graph convolutional network to embed the knowledge graphs into the low-dimensional space. Second, GRL employs both GAN and long short-term memory (LSTM) to record trajectory sequences obtained by the agent from traversing the knowledge graph and generate new trajectory sequences if needed. At the same time, GRL applies domain-specific rules accordingly. Finally, GRL employs the deep deterministic policy gradient method to optimize both rewards and adversarial loss. The experiments show that GRL is able to both generate better policies and outperform traditional methods for several tasks.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Knowledge graph is designed to represent the various entities or concepts and their relations existing in the real world. It constitutes a huge semantic network graph in which nodes represent entities or concepts, and edges represent attributes or relations. Knowledge graph has been used to represent a variety of large-scale knowledge bases. However, the real-world knowledge bases are usually incomplete [1,2], one needs to complete it by inferring the missing ones with the help of existing information. After that, one can apply the completed knowledge base to real tasks, such as question and answer and searching engines. This knowledge base completion process is known as the knowledge graph completion problem [3–5].

Existing knowledge graph completion methods can be classified into three types: (1) Path ranking algorithm (PRA) based [6]. They predict the potential relations between entities by connecting existing paths of entities; (2) Representation learning based. They first map the entities and relations into the low-dimensional space, and then reason through vector operations, e.g., TransE [7];

and (3) Probability graph based [8] includes the Markov logic networks and their derivatives, e.g., reinforcement learning methods [9]. These methods have been shown to perform well in their respective areas of expertise.

However, most existing methods did not consider the fact that the knowledge graph can generate new sub-graphs (including nodes and edges). The previous reinforcement learning methods assume that the agent can find the target entity in the knowledge graph within limited time steps [10–12]. However, the target entity may be missing and thus requires completion. For example, the question “who is the current president of USA?” is formalized as the triple (“USA”, “president of”, ?). If there is a complete triplet (“USA”, “president of”, “Trump”) in the knowledge graph, the agent will find the target entity “Trump” easily. However, if the target entity is missing in the knowledge graph, how could the agent find the correct answer? Inspired by the graph generation techniques [13], we can generate the sub-graphs when the target entity is not found in the known knowledge graph, and let the agent select the correct edges and target entity. Knowledge graph completion is essentially a problem of generating missing parts based on the existing. Compared with the previous methods, we also inferred the missing parts with the help of existing ones in the knowledge graph, but we use the

* Corresponding author.

E-mail address: qiawang1101@163.com (Q. Wang).

graph generation techniques to strengthen the generation ability and optimize the reward so that the agent can find the target entities or missing entities more intelligently. Further, in existing reinforcement learning based models, the sampling method faces large variance and relatively low learning efficiency, and thus faces the challenges of fragile convergence and sparse rewards.

Deep neural network has shown its great flexibility and power by learning to represent the world as a nested hierarchy of concepts, achieving excellent results in lots of applications [14]. It should be pointed out that when using deep neural networks, very little work directly uses a single neural network to encode the graph structure, but combines multiple different forms of neural networks to enhance the capabilities of the entire network architecture. Motivated by these, in this paper, we propose a new framework based on GAN [15] and reinforcement learning for knowledge graph completion, named GRL. We divide the knowledge graph completion problem into two parts: first, the target entity can be found within the limited time step from the original knowledge graph; second, the target entity cannot be found from the original knowledge graph while there are still time steps left, one can generate a new sub-graph as shown in Fig. 1. In other words, when the agent did not find an answer within the time steps, it can generate a new path at the right time, which refers to the time when the agent thinks that generating a new path or node is more conducive to find the target entity based on the rules. This mechanism is more intelligent than the previous work. As reinforcement learning method learns policies through interaction with the environment to guide the agent to walk in the knowledge graph [10,11], it is a good fit for sequence decision-making problems. As a result, we define the knowledge graph completion problem as a Markov process, and explore the rules that can be introduced in both state transition process and rewards, which can better guide the walking path under the optimization of GAN. We employ long short-term memory (LSTM) [16] as a generator of GAN, which not only records historical trajectories but also generates new sub-graphs, and trains policy networks with GAN. Further, to better generate new sub-graphs, a graph neural network (GNN) [17] is used to embed the knowledge graph into low-dimensional vectors and parameterize the message passing process at each layer. Finally, we employ DDPG [18] to optimize the entire model.

To summarize, our main contributions are:

- We are the first to introduce the graph generation techniques to knowledge graph reasoning to help agents better find the target entity.
- We are the first to use GAN to optimize reward in the knowledge graph completion problem, in which the introduced rules are optimized to guide the agent's walking path and combined with trajectory generation to train the policy network.
- DDPG is employed in the framework to solve the problems of sparse rewards and low sampling efficiency brought by REINFORCE in previous work. The experimental results show the efficiency and effectiveness of our model.

2. Related work

Knowledge graph completion methods are mainly divided into three categories: representation learning based, path ranking based, and reinforcement learning based.

Representation learning aims to represent the semantic information of the research object as a dense low-dimensional real-value vector, while knowledge representation learning [19] is representational learning of entities and relations in the knowledge graph [20]. We can use modeling methods to represent entities and vectors in low-dimensional dense vector space, and

then perform calculations and reasoning [21]. Translation-based models are representatives of knowledge representation learning, and TransE is the most classic model [7]. TransE projects both entities and relations into a continuous low-dimensional vector space, interpreting relations as translating operations between head and tail entities. The translation assumption in TransE implies the equation that $h + r \approx t$. Many variants have been derived based on TransE, such as TransH [22], TransR [23], and TransG [6], which have improved and surpassed the ontology and implemented state-of-the-art in their respective areas of expertise. Due to the powerful representation capabilities of deep neural networks, they have recently been used in graph structure data representation, including single layer model, neural tensor networks, convolutional 2D, etc. There are also methods such as the RESCAL factorization model, structured embedding, which can also make full use of the existing structured information of the knowledge graph.

The basic idea of the path ranking algorithm (PRA) [24] is to use a path to connect two entities as a feature to predict the relationship between two entities. In the knowledge graph, a path refers to a sequence linking two entities. The learning phase is separated into three parts: feature extraction, feature calculation, and construction classifier. The common methods for feature extraction in the PRA algorithm are random walks, breadth-first search, and depth-first search. The common method of eigenvalue calculation is random walk probability. The training methods of classifiers mainly include single-task and multi-task learning methods [25]. Recent variants of single-task methods include incorporating a text corpus as additional evidence during reasoning and importing better schemes to generate more predictive paths [26,27]. A key factor of multi-task learning is to model the construction of task connection [28], via either feature sharing [10] or parameter sharing [29].

In recent years, there is an increasing attention of using reinforcement learning to solve the pathfinding problem. That is, given the source entities and relations, one looks for another entity through the path [30]. This can be defined as a Markov decision process (MDP). Deep-path is the first model to use the reinforcement learning method to perform path query in the knowledge graph, which has an important inspiration for the subsequent derivative models [11]. However, it focuses more on the search for paths like traditional methods, and it has to know the target entity in advance. The MINERVA model enhances the role of reinforcement learning based on deep-path [12]. It does not need to know the target entity in progress but instead causes the agent to "walk" on the knowledge graph until it finds the answer. There are certain problems in existing methods, for example, the agent has reached a correct answer, but no linking path points to the source entity in training, which leads to zero reward. Or there is no correct path in training, but the agent happens to arrive at the correct answer. Therefore, Lin et al. uses the action dropout method effectively to improve the above problems [31]. M-WALK uses RNN to record all the sequence trajectories obtained by the agent traversal and then iterate over-optimization policy with using the Monte Carlo sampling and Q-learning algorithms to learn the model [12]. Compared with the previous methods that are based on reinforcement learning, we add GAN [32] to help LSTM to generate and optimize the historical trajectory sequence. Inspired by graph generation, we can make the agent intelligently choose whether to continue to find the target node on a given graph or when it is appropriate, a new sub-graph can be generated to help the agent find the target node more efficiently.

The sequence generation problem of graphs has recently received many attentions and many recent works have been proposed, such as SEQGAN [13], BGAN [33], ORGAN [34], and MALIGAN [35]. These methods make full use of GAN's powerful generation capabilities [36]. We believe that generating new graphs

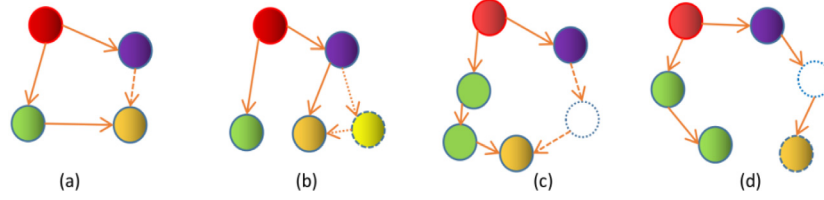


Fig. 1. The graph generation process is divided into four cases: (a) from the initial node (red circle) to the target node (golden circle), a path (dashed line) can be generated at the current node (purple circle); (b) the target node has become obsolete and generated a new correct node (yellow circle) to replace the original node; (c) generate new nodes (open circle) and paths to the target node at the current node; (d) generate new nodes at the current node and paths until a new target entity (golden circle with dotted frame) is generated.. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

from known graph structures and outputting them in the form of sequences has crucial implications for the completion of knowledge graphs. It allows us to not only find the answer from the original graph but also automatically reason or generate new entities and paths to help us find the desired answers when the answers are not found in the original graph.

There are once some works involving time-aware dynamic networks for link prediction. For example, Esteban et al. design a model of the relationship between time to boost time-aware link prediction [37]. Loglisci et al. propose a multi-way neural network architecture for modeling event-based relational graph [38]. Trivedi et al. present a model to capture temporal auto-correlation of the data to improve the performance of link prediction [39].

In our work, we focus on the static knowledge graph, where static refers to the used data comes in a static way. Although our entire framework uses “dynamic” graph generation techniques and LSTM to process “dynamic” time steps (sequence decisions that change over time), we still use traditional static data for knowledge completion, such as WN18RR, FB15k-237, and NELL-995. In short, our method is dynamic, but works for static knowledge graphs.

3. Methodology

3.1. Problem definition

We formally define the research problem of this paper in the following. The knowledge graph can be defined as $G = \{E, R\}$, where E represents the set of all entities (e) and R represents the set of all relations (r) that exist in the knowledge graph or newly generated entities or relations. In the applications such as searching and Q&A, most problems are to infer another entity when we know an entity and relation, which can be formed by the triple in the knowledge graph as $(e_s, r_q, ?)$, where e_s represents the starting entity of the query process and r_q represents the relational paths searching, and “?” indicates the target entity e_T that needs to be found and $\{e_s, e_T\} \subseteq E$, $\{r_q\} \subseteq R$. It should be emphasized that in the query task, if the target entity e_T exists in the knowledge graph, it can be directly found; otherwise, it needs to be found by inferential generation of sub-graph paths. Most existing methods solve the link prediction problem by finding the relations between two known entities, or determining whether given triples are correct. Obviously, our research problem is more exploratory and challenging because we cannot only reason in the original graph, but sub-graphs can be automatically generated to extend the relational path until the target entities are found. The complete process of the GRL framework is shown in Fig. 2.

3.2. Markov decision process

Reinforcement learning is a simple framework for learning from interactions to achieve the desired goals. Learners and

decision-makers are called agents, and those interacting with agents are called environments. As the interaction continues, the agent chooses actions to perform, and the environment reacts to the performed actions, and moves the agent to a new environment. At the same time, the environment also produces returns and the agent attempts to maximize these returns over a time period.

Reinforcement learning can be used for knowledge graph completion because it satisfies four necessary conditions: environment, state, action, and reward. We formalize the process of iterative reasoning in the knowledge graph into the Markov decision process $M = (S, A, P, R)$, where $S = \{s_i\}$ is the state set, $A = \{a_i\}$ is the action set, P is the state transition probability, and R is the reward function. The traversal process of the agent on the graph can be seen as generating a trajectory sequence $(s_0, a_0, r'_0, \dots, s_t, a_t, r'_t, \dots, s_T, a_T, r'_T)$, which represents the states, actions, and rewards of the agent at each time step, where T is the maximum time steps to reach the target entity.

In the following, we would identify the reinforcement learning problem by representing the individual agent and the environment in detail.

State. The state $s \in S$ is a representation of the environment and the state space S which includes any available combination of both entity space and relation space ($E \times R$). Since the entities and relations in the knowledge graph are both discrete and atomic which cannot be used directly for modeling, we need to map them to a low-dimensional contiguous space for computation using the graph neural network (GNN) [11]. The specific embedding process will be discussed in detail in Section 3.3. The state is initialized to $s_0 = \{e_s, r_q, R_{e_s}, E_{e_s}\}$, where R_{e_s} represents the set of relations (edges) connecting to the initial entity e_s , and E_{e_s} represents the set of entities connecting to the initial entity e_s . The state at step t is $s_t = s_{t-1} \cup \{a_{t-1}, e_t, R_{e_t}, E_{e_t}\}$, which indicates that the action a_{t-1} is performed at the last time step ($t-1$), and the state changes from s_{t-1} to s_t , while the agent is accessing the entity e_t at time step t . s_T represents the final state, at which point the agent finds the target entity e_T .

Action. The action a_t is a representation of the behavior choice of an agent at time step t . The action space A refers to a collection of the actions, and A_t indicates the set of all feasible actions at time t , $A_t \cong R_{e_s}$ and $A_t \subseteq A$. The agent can select its neighboring relational path to another neighboring entity at each state s_t . The choice is to select the optimal path according to the policy network until it reaches the target entity e_T . In this process, the agent can also select “stop”, when the “stop” is selected, the state s_t at this moment can be used as a prediction of the final target $e_T \approx e_t$.

State Transition. The behavior changing of an agent at each time step t can be seen as a state transition distribution shown in Eq. (1), where $p(a_t | s_t, \dots, s_0)$ is displayed as a policy network π_θ .

$$p(s_{t+1} | s_t, \dots, s_0) = \sum_{a_t} p(a_t | s_t, \dots, s_0) p(s_{t+1} | s_t, \dots, s_0, a_t) \quad (1)$$

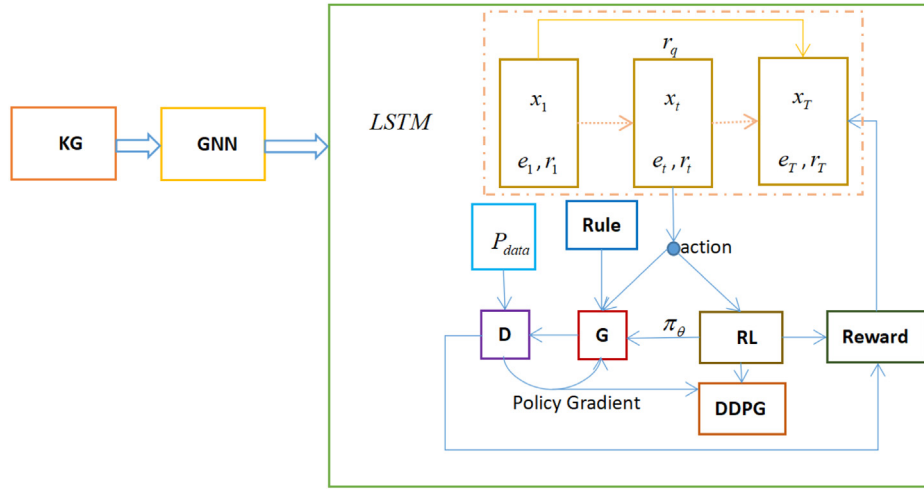


Fig. 2. The architecture of our GRL framework. The agent can choose different actions if needed: continue walking (RL) or generate sub-graph (GAN). First, KG is pre-processed by GNN as an input (embedding into a low-dimensional space, message passing, and aggregation at different levels), and then LSTM maps the embedding representation sequence into a series of hidden states, and recursively generate a new sub-graph embedding vector, and at the same time form a policy to guide the agent to take actions. At this time, the agent will choose whether to directly perform reinforcement learning optimization or to generate a sub-graph according to the environment in which it is located. Finally, the entire framework is optimized through reinforcement learning, and it will be rewarded as long as the target entity is finally found.

Specifically, the state transition probability $p(s_{t+1} | s_t, a_t)$ is deterministic and known, which indicates the probability that the environment transitions to the state s_{t+1} at the next moment after the agent makes an action a_t in line with the current state s_t , the set of entities E_s , and relations R_s that are connected to e_t at this time are known. Therefore, domain rules can be introduced during the state transition to guide the next choice of the agent or the direction of the state transition, the environment carries through the actions that comply with the given rules, then the inviable actions presented by the policy network will be rejected and the state will remain unchanged. The state transition process can also be described as the state set interacting with the action set to produce the next state set: $S \times A \rightarrow S$.

Rewards. After the agent takes an action under a certain state if the obtained new state is closer to the target entity or more favorable to the target entity, a favorable return value 1 is obtained, and conversely, an unfavorable result 0 is obtained. The function that determines what kind of return value should be based on the specific situation is the reward function. Under this decision process, the return value of each step in each path from the initial state to the termination state is summed, and the path with the largest return value is the optimal path. The decision made by the agent at the moment will definitely maximize the future return.

In order to make the agent walk more in accordance with the rules, the extended path and sub-graph more in line with the introduced rules, and the policy better optimized under the joint training of GAN and RL, we employ Wasserstein-GAN [40,41] to define adversarial rewards $V(\pi_\theta, D_\psi)$:

$$\min_{\theta} \max_{\psi} V(\pi_\theta, D_\psi) = E_{x \sim p_{data}} [D_\psi(x)] - E_{x \sim \pi_\theta} [D_\psi(x)] \quad (2)$$

Here, the policy π_θ acts as a generator of GAN, parameterized by θ . D_ψ is the discriminator network, parameterized by ψ . x represents the input knowledge graph, and p_{data} represents the underlying data distribution, which defines the final trajectory sequence or the trajectory sequence of the intermediate state (both the rewards). We will match the rewards with the original rewards (0 and 1) to optimize the policy by a reinforcement learning algorithm.

3.3. Graph neural networks

Graph neural network (GNN) [17] is a novel neural network model, which is based on expanding a neural network method for the sake of processing data in the form of graph structures. The GNN can deal with different types of graphs (e.g., acyclic, cyclic, directed, and undirected). It maps a graph and its nodes into a d-dimensional Euclidean space to learn the final classification/regression model, and the GNN parameterizes an uncoiling of the iterative message passing procedure.

In this work, the GNN [42] employed is made up of many layers, which is indicated by $l \in N$ with each layer jointed with a dimensionality k_l and an activation function ϕ (such as ReLU, tanh, sigmoid). According to past experiences, we generally use the sigmoid function in the output layer and the tanh function in the hidden layer. However, in practical applications, tanh and sigmoid will tend to saturate at the end value and cause the training speed to become slow, but ReLU performs well, so we generally use the ReLU function by default in deep networks. Above the input graph x , each layer $l \in N$ of the GNN regards the activations of the previous layer $H(l-1) \in R_n \times k_{l-1}$ as an input.

The ultimate idea is to perform message passing over each edge type for a total of L layers. At the $(l-1)th$ layer of the GNN, we aggregate all messages from different edge types to compute the next layer node embedding $H(l)$, we employ Agg function to represent an aggregation function which can be one of {max, mean, sum, concat} [43]. The weight parameters are matrix $W \in R_{d_{l-1} \times d_l}$, optional bias parameters are $B \in R_{d_{l-1} \times d_l}$, a supply of linear mutations are $F: R_{n \times n} \rightarrow R_{n \times n}$, and $H^{(0)} = x$. Specifically, the hierarchical propagation in GNN is as follows:

$$H^{(l)} \leftarrow \text{Agg}(\phi(B + \sum_{f \in F} f(x) H^{(l-1)} W)) \quad (3)$$

We can also choose to employ one of GNN's concrete network structures, such as using a graph convolutional network (GCN) [44], and the propagation process is as follows:

$$H^{(l)} \leftarrow \text{Agg}(\phi(B + \tilde{D}_i^{-\frac{1}{2}} \tilde{A} \tilde{D}_i^{-\frac{1}{2}} W)) \quad (4)$$

Here $\tilde{A} = D^{-\frac{1}{2}} x D^{-\frac{1}{2}}$ the symmetric diagonalization of x given the diagonal degree matrix D , and $\tilde{A} = x + I$, $\tilde{D} = \sum_k \tilde{A}_k$. We

employ L layer GCN to compute the final node embedding matrix $X = H^{(L)}$.

3.4. Generative model and policy network

The policy is the mechanism that determines the individual behavior of an agent, it is a mapping from the state (state space) to behavior (action space) $\pi : S \rightarrow A$, either deterministic or stochastic.

Knowledge graph completion is a dynamic process, so we employ the LSTM [45] as a generation model which can catch the underlying structures in the sequences with time irregularities, and effectively prevent gradient explosions compared to RNN. We define a recursive history sequence $h_t = (h_{t-1}, A_{t-1})$, A_{t-1} represents the actions before t steps that have been taken. LSTM maps the input embedding representations sequence $x_1, x_2, \dots, x_t, \dots, x_T$ of x into a sequence of hidden states $h_1, h_2, \dots, h_T$, and generate a new sub-graph embedding vectors recursively as follows, where the parameters are a bias vector b and a weight matrix W .

$$h_t = \text{LSTM}(h_{t-1}, x_{t-1}) \quad (5)$$

$$p(x_t | x_1, \dots, x_t) = \text{soft max}(b + Wh_t) \quad (6)$$

According to the historical trajectory, the policy network chooses to select an action from all available action spaces, then takes it as input and parameterized by the network into a two-layer feed-forward network with Φ activation function (ReLU). That receives the embedding x_t and current history representation h_t and outputs a probability distribution for possible actions in which discrete actions are sampled. All actions are embedded in action space A_t , and the policy network can be defined as follows:

$$\pi_\theta(a_t | s_t) = \text{soft max}(A_t \times W_2 \Phi(W_1[x_t; h_t])) \quad (7)$$

Where the weight W_1 and W_2 are the weight parameters of the LSTM, and the θ is the parameter of the policy network formed by the embedding matrix X .

3.5. Deep deterministic policy gradient training

The policy gradient approach is to construct the goal of maximizing rewards as a minimization of the loss function $L(\theta)$, where θ (including θ^π and θ^Q) is the parameterize agent. Most of the previous work used REINFORCE [30], which is a gradient-based optimization method. But one of its main drawbacks is that the variance between different paths during training is enormous, resulting in unstable training, and this is a common problem in using Monte Carlo methods in high-dimensional space.

We employ one of state-of-the-art policy gradient methods: deep deterministic policy gradient (DDPG) [18], which is a model-free algorithm for processing continuous high-dimensional action space. DDPG employs an actor-critic architecture [46] sustaining an action-value function approximation (critic) and a deterministic policy (actor). The Actor-Critic Algorithm is a reinforcement learning method that combines policy gradient and Temporal-Difference learning. The actor is the policy function $\pi_\theta(s_t, a_t)$ to learn one policy is to get the highest possible reward. The critic represents the value function $V_\phi(s_t)$, which estimates the value function of the current policy to evaluate the actor's quality. With the help of the value function, the Actor-Critic algorithm can update the parameters in a single step without waiting for the end of the round to update. Actors and critics are parameterized by deep neural networks with θ^π and θ^Q . The policy gradient for the sampling of the actor's parameter θ^π is calculated by the backpropagation of the combined actor and critic networks. After each action of the agent, the current state, the action of

Table 1
Statistics of the datasets.

Dataset	Entity	Relation	Test	Train	Avg. degree	Median degree
WN18RR	40943	11	3134	86835	2.19	2
NELL-995	75492	200	3992	154213	4.07	1
FB15K-237	14541	237	20466	272115	19.74	14

the actor, the observed reward, and the next state are contained in $(s_t, a_t, r'_t, s_{t+1})$ and saved into a cyclic replay buffer R . The actor and critic networks are renovated by randomly sampling mini-batches from R .

The critic is trained by minimizing the loss function:

$$L = \frac{1}{T} \sum_t (y_t - Q(s_t, a_t | \theta^Q))^2 \quad \text{where} \quad (8)$$

$$y_t = r'_t + \gamma Q'(s_{t+1}, \pi'(s_{t+1} | \theta^{\pi'})) | \theta^{Q'}$$

The actor is trained using the sampled policy gradient:

$$\nabla_{\theta^\pi} J \sim \frac{1}{T} \sum \nabla_a Q(s, a | \theta^Q) |_{s=s_t, a=a_t} \nabla_{\theta^\pi} \pi(s | \theta^\pi) |_{s=s_t} \quad (9)$$

4. Experiments

In this section, we describe the experiments in detail to verify the effectiveness and efficiency of our model. We first describe the data sets and parameter settings we use in the experiment, and then we design a series of experiments, such as link prediction, fact classification, ablation study, and sensitivity analysis. We prove that our model's overall performance is better than the traditional embedding-based methods, random walk methods, and other baseline methods, especially in generalization and transference of rules.

4.1. Data sets and settings

The experiments are based on three data sets, namely WN18RR [47], FB15k-237 [48], NELL-995 [49], which are commonly used in existing knowledge graph completion methods. The detailed description is given in Table 1. We also employ AMIE+ [50] to automatically extract Horn clause rules from these datasets, and take the training set as input. We consider rules with length not longer than 2 and confidence levels not less than 0.8 in order to achieve efficient extraction. The length of a Horn clause rule is the number of atoms present to its precondition, e.g., $\forall x, y: (x, \text{WorkInCompany}, y) \Rightarrow (x, \text{IsEmployedby}, y)$ has the length of 1. Setting the confidence threshold to 0.8 allows these data sets to reach optimal performance.

Next we give details of the parameter settings and implementation details in the experiment. Our framework is deployed in TensorFlow and can be trained and optimized end-to-end using gradient methods. We employ two convolutional layers with 10 and 20 kernels, 2×2 kernel sizes and a stride of 1 to embed the input KG and a two-layer LSTM with a hidden dimension of 200 as the path encoder. We set the embedding dimensions of entities and relations to 8 and 64 respectively. The maximum path lengths T of the agent travels on the data sets are set to 20, and the discount rate γ is set to 0.5. We use the Adam optimizer [51] which has learning rate α of 0.0001, a mini-batch size of 128 to optimize the model. We use the same hyperparameter setting for each task in the experiment.

Table 2

The results of link prediction(all data was obtained under our implementation settings)(%).

Dataset metric	WN18RR		FB15K-237		NELL-995	
	MRR	Hits@10	MRR	Hits@10	MRR	Hits@10
TransE	38.2	89.2	32.3	47.1	31.2	50.8
TransH	37.5	86.7	33.5	64.4	35.2	61.2
TransR	39.6	92.0	36.4	68.7	46.4	63.5
ConvE	77.3	74.7	64.1	70.0	74.7	86.4
PRA	83.8	81.9	72.6	73.9	37.1	49.7
MINERA	86.2	89.5	85.7	80.3	72.5	83.1
M-Walk	90.5	91.2	88.4	82.2	75.2	85.2
GRL	93.7	93.5	95.4	91.2	79.4	88.6

4.2. Link prediction

Link prediction is an important measure of knowledge graph completion. When given entities and relations, the model predicts other missing entities. More specifically, in this task, we predict * in given $(h, r, *)$ or in given $(*, r, t)$. Note that link prediction is a measure of many AI tasks, such as knowledge representation learning and relation extraction.

For each test triple (h, r, t) in the knowledge graph, we replace the head entity h by every entity h' in the dictionary, and calculate the true value for the corrupted triple (h', r, t) . Ranking the truth values in descending or ascending order, we get the rank of the correct entity h . Similarly, we can get another rank by corrupting the tail entity t . Converging all the test triples, there are two metrics for evaluation: the mean reciprocal rank (MRR) and the proportion of testing triple whose rank is not larger than 10 (Hits@10).

We performed link prediction experiments with FB15k-237, WN18RR, NELL-995 data sets and compared them with some representative methods. From the experimental results in Table 2, we can see that our model performs better on the three data sets than the previous method. Compared with traditional embedding-based methods, our approach introduces GNN and message pass, which makes embedding more efficient and semantic information content more complete. Compared to preceding reinforcement learning methods, we introduced rules that make agents more directed and efficient in finding paths. A better policy can also be generated combining of GAN and DDPG algorithms.

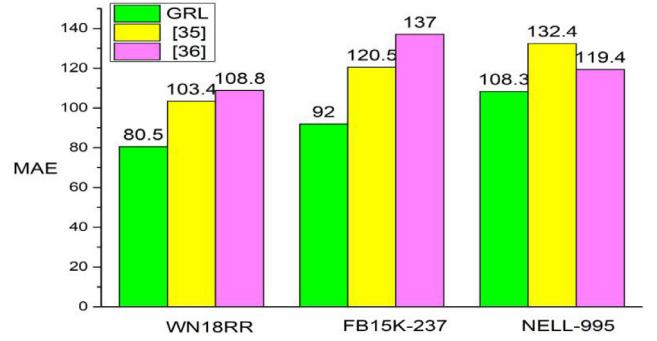
4.3. Fact classification

The fact classification task is to prove the ability of the model to judge the true and fault of the triples, and it is intended to predict whether the given triple (h, r, t) is correct. It is the classic task of knowledge reasoning. We also use FB15k-237, WN18RR, NELL-995 as the benchmark data set for this task. Note that the classification assessment requires negative samples, but the datasets already provide negative triples. Instead of ranking the target entities, this task directly ranks all the positive and negative samples for a particular relation. Table 3 shows the overall results of all the methods. Our model gets even better results on this task, which proves GRL's ability to judge the correctness of the triples in the knowledge graph, which also proves the effectiveness and efficiency of the model in reasoning from another aspect, because GRL introduces the mechanism of GAN (GAN itself has the ability to identify authenticity) to optimize the model so that it can adaptively generate the correct node to replace the wrong or missing node. In addition, due to its message passing mechanism, GNN also makes the model have a global awareness of the knowledge graph, which makes the model more inferential.

Table 3

The results of fact classification(MAP)(all data was obtained under our implementation settings).

Dataset	WN18RR	FB15K-237	NELL-995
TransE	0.277	0.383	0.439
TransH	0.302	0.394	0.534
TransR	0.322	0.407	0.603
ConvE	0.355	0.443	0.642
PRA	0.364	0.524	0.762
MINERA	0.378	0.675	0.745
M-Walk	0.477	0.769	0.795
GRL	0.590	0.862	0.873

**Fig. 3.** Time prediction (in hours).

Time prediction. Our method has dynamic elements, because it can adaptively generate new sub-graphs, which requires some comparison with some dynamic methods. It is also a fact prediction, so we predict the prospective value of the next time the fact (e_s, r_q, e_t) could occur by the given test triplet (e_s, r_q, e_t) . The expectation and calculation formula is specified in [52]. The result is the Mean Absolute Error (MAE) between the predicted time and true time in hours, as Fig. 3 shows. We can see that our model performs better than some of the previous dynamic network prediction baselines [38,39], which indicates our model can practically capture non-linear relational and temporal dynamics and our framework has significantly better performance for link prediction than baselines that does not consider temporal non-linear dynamics.

4.4. Sensitivity analysis

As a model based on GAN, GLR has many hyperparameters, but in fact most of them are not important for performance because they are within reasonable limits (e.g. $H > 30$). An exceptional case depends on the length T of the random walk. We use WN18RR to study the change in the model's performance under different path lengths in the link prediction task, so as to judge the path length when the model performs optimally. The model run five times at different lengths and then the average value is as showed in Fig. 4. We experimentally prove that the longer the path length, the better the GRL performs, and compared to the previous methods (the length of paths does not exceed 10), we can complete the task in a longer path (some parts are the generated paths), which also proves the validity of our model in reasoning and generating.

4.5. Ablation study

The ablation study is utilized to demonstrate the effectiveness and efficiency of the various components of the model. In the fact classification experiment, we removed WGAN, and the experimental results of GRL on the WN18RR, FB15K-237, and NELL-995

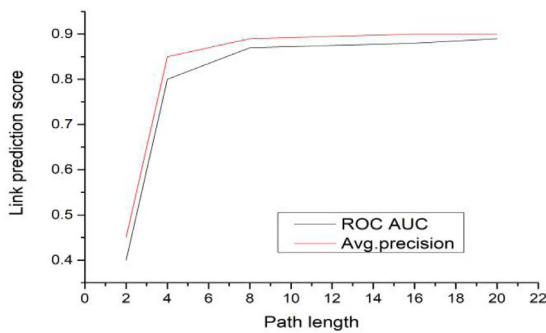


Fig. 4. The effect of path length on model performance.

Table 4

Results of ablation study. In the fact prediction experiment, the GRL after the WGAN, GNN, and DDPG is removed or replaced separately and compared with the original GRL (all data was obtained under our implementation settings).

Dataset	WN18RR	FB15K-237	NELL-995
WGAN(Remove)	0.531	0.733	0.725
GNN(Replace)	0.537	0.776	0.803
DDPG(Replace)	0.531	0.784	0.768
GRL(Original)	0.590	0.862	0.873

data sets were reduced by 10%, 15% and 17%, respectively, indicating that GAN plays an important role in the GRL and can produce better rewards and train a better policy. Then, we use TransE to replace the GNN in the GRL to embed the knowledge graph, the experimental results on the WN18RR, FB15K-237 and NELL-995 data sets were reduced by 9%, 10% and 8%, respectively, which shows the powerful representation and generalization ability of GNN, and has some implications for the study of knowledge graph representation learning. Finally, we replaced DDPG with the REFORINCE method, and the experimental results on the WN18RR, FB15K-237, and NELL-995 data sets were decreased by 10%, 9%, and 12%, respectively (see Table 4).

5. Conclusion and future work

We propose a novel framework for knowledge graph completion. We observe that the dynamics in the knowledge graph are important factors for knowledge graph completion. We then design a set of a neural network architecture based on GAN and reinforcement learning. We use WGAN to introduce rules to better guide agents to find paths, dynamically record, and generate sub-graph sequences, and form policies in conjunction with LSTM. Finally, we utilize DDPG methods to optimize the policy. Experiments demonstrate that our model performs well in reasoning and generation tasks. In the future, we plan to apply Neural Architecture Search (NAS) to knowledge graph reasoning, and combine GAN and reinforcement learning to design a more powerful neural network architecture.

CRediT authorship contribution statement

Qi Wang, Yuede Ji, Yongsheng Hao and Jie Cao conceived and designed the study, and conducted experiments. Qi Wang wrote the paper, and Yuede Ji, Yongsheng Hao and Jie Cao reviewed and edited the manuscript. All authors read and approve the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work was partly supported by the Chinese National Funding of Social Sciences (No. 16ZDA054), the National Natural Science Foundation of China (NSFC) under grant (NO. 41475089).

References

- [1] Y. Zhang, H. Dai, Z. Kozareva, A.J. Smola, L. Song, Variational reasoning for question answering with knowledge graph, in: 32nd AAAI Conf. Artif. Intell. AAAI 2018, 2018, pp. 6069–6076.
- [2] Variational knowledge graph reasoning, in: NAACL HLT 2018–2018 Conf. North Am. Chapter Assoc. Comput. Linguist. Hum. Lang. Technol. - Proc. Conf. Vol. 1, 2018, pp. 1823–1832, <http://dx.doi.org/10.18653/v1/n18-1165>.
- [3] Théo Trouillon, et al., Knowledge graph completion via complex tensor factorization, *J. Mach. Learn. Res.* 18 (1) (2017) 4735–4772.
- [4] Jianfeng Gao, Michel Galley, Lihong Li, Neural approaches to conversational ai, *Found. Trends® Inf. Retr.* 13 (2–3) (2019) 127–298.
- [5] Ni Lao, William W. Cohen, Relational retrieval using a combination of path-constrained random walks, *Mach. Learn.* 81 (1) (2010) 53–67.
- [6] Ni Lao, William W. Cohen, Cohen relational retrieval using a combination of path-constrained random walks, *Mach. Learn.* 81 (1) (2010) 53–67.
- [7] Antoine Bordes, et al., Translating embeddings for modeling multi-relational data, *Adv. Neural Inf. Process. Syst.* (2013).
- [8] Geoffrey Hinton, et al., Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups, *IEEE Signal Process. Mag.* 29 (6) (2012) 82–97.
- [9] H. Wang, S. Li, R. Pan, M. Mao, Incorporating graph attention mechanism into knowledge graph reasoning based on deep reinforcement learning, in: EMNLP-IJCNLP 2019–2019 Conf. Empir. Methods Nat. Lang. Process. 9th Int. Jt. Conf. Nat. Lang. Process. Proc. Conf, 2020, pp. 2623–2631, <http://dx.doi.org/10.18653/v1/d19-1264>.
- [10] Wenhan Xiong, Thien Hoang, William Yang Wang, DeepPath: A reinforcement learning method for knowledge graph reasoning, 2017, arXiv preprint [arXiv:1707.0669](https://arxiv.org/abs/1707.0669).
- [11] Rajarshi Das, et al., Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning, 2017, arXiv preprint [arXiv:1711.05851](https://arxiv.org/abs/1711.05851).
- [12] Yelong Shen, et al., M-walk: Learning to walk over graphs using monte carlo tree search, *Adv. Neural Inf. Process. Syst.* (2018).
- [13] R. Devon Hjelm, et al., Boundary-seeking generative adversarial networks, 2017, arXiv preprint [arXiv:1702.08431](https://arxiv.org/abs/1702.08431).
- [14] Kim Yoon, Convolutional neural networks for sentence classification, 2014, arXiv preprint [arXiv:1408.5882](https://arxiv.org/abs/1408.5882).
- [15] Matthew B.A. McDermott, et al., Semi-supervised biomedical translation with cycle wasserstein regression GANs, in: Thirty-Second AAAI Conference on Artificial Intelligence, 2018.
- [16] Inci M. Baytas, et al., Patient subtyping via time-aware LSTM networks, in: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2017.
- [17] Timothy P. Lillicrap, et al., Continuous control with deep reinforcement learning, 2015, arXiv preprint [arXiv:1509.02971](https://arxiv.org/abs/1509.02971).
- [18] Richard S. Sutton, Andrew G. Barto, *Reinforcement learning: An introduction*, MIT press, 2018.
- [19] Ruobing Xie, Zhiyuan Liu, Maosong Sun, Representation learning of knowledge graphs with hierarchical types, in: IJCAI, 2016.
- [20] L. Yao, C. Mao, Y. Luo, KG-BERT: BERT for knowledge graph Completion, 2019, [Online]. Available: <http://arxiv.org/abs/1909.03193>.
- [21] Tinghuai Ma, Shao, Wenye, Hao, Yongsheng, Graph classification based on graph set reconstruction and graph kernel feature reduction, *Neurocomputing* (2018).
- [22] Zhen Wang, et al., Knowledge graph embedding by translating on hyperplanes, in: Twenty-Eighth AAAI cOnference on Artificial Intelligence. 2014.
- [23] Guoliang Ji, et al., Knowledge graph embedding via dynamic mapping matrix, in: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), 2015.
- [24] Matt Gardner, et al., Improving learning and inference in a large knowledge-base using latent syntactic cues, in: Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing. 2013.
- [25] H. Rong, T. Ma, J. Cao, et al., Deep rolling: A novel emotion prediction model for a multi-participant communication context, *Inform. Sci.* 488 (2019) 158–180.
- [26] Matt Gardner, et al., Incorporating vector space similarity in random walk inference over knowledge bases, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). 2014.

- [27] Rie Kubota Ando, Tong Zhang, A framework for learning predictive structures from multiple tasks and unlabeled data, *J. Mach. Learn. Res.* (2005) 1817–1853.
- [28] Q. Wang, Y. Hao, ALSTM: An attention-based long short-term memory framework for knowledge base reasoning, *Neurocomputing* 399 (2020) 342–351, <http://dx.doi.org/10.1016/j.neucom.2020.02.065>.
- [29] Jingrui He, Yan Liu, Qiang Yang, Linking heterogeneous input spaces with pivots for multi-task learning, in: *Proceedings of the 2014 SIAM International Conference on Data Mining, Society for Industrial and Applied Mathematics*, 2014.
- [30] Q. Wang, Y. Hao, J. Cao, ADRL: An attention-based deep reinforcement learning framework for knowledge graph reasoning, *Knowledge-Based Syst.* 197 (2020) 105910, <http://dx.doi.org/10.1016/j.knosys.2020.105910>.
- [31] Xi Victoria Lin, Richard Socher, Caiming Xiong, Multi-hop knowledge graph reasoning with reward shaping, 2018, arXiv preprint [arXiv:1808.10568](https://arxiv.org/abs/1808.10568).
- [32] Ian Goodfellow, et al., *Generative adversarial nets*, *Adv. Neural Inf. Process. Systems* (2014).
- [33] Tong Che, et al., Maximum-likelihood augmented discrete generative adversarial networks, 2017, arXiv preprint [arXiv:1702.07983](https://arxiv.org/abs/1702.07983).
- [34] Zonghan Wu, et al., A comprehensive survey on graph neural networks, 2019, arXiv preprint [arXiv:1901.00596](https://arxiv.org/abs/1901.00596).
- [35] Gabriel Lima Guimaraes, et al., Objective-reinforced generative adversarial networks (organ) for sequence generation models, 2017, arXiv preprint [arXiv:1705.10843](https://arxiv.org/abs/1705.10843).
- [36] Lantao Yu, et al., Seqgan: Sequence generative adversarial nets with policy gradient, in: *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.
- [37] Esteban Cristóbal, et al., Predicting the co-evolution of event and knowledge graphs, in: *2016 19th International Conference on Information Fusion (FUSION)*, IEEE, 2016.
- [38] Corrado Loggisci, Donato Malerba, Leveraging temporal autocorrelation of historical data for improving accuracy in network regression, *Stat. Anal. Data Min.: ASA Data Sci. J.* 10 (1) (2017) 40–53.
- [39] Rakshit Trivedi, et al., Know-evolve: Deep temporal reasoning for dynamic knowledge graphs, in: *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, JMLR. org, 2017.
- [40] Huidong Liu, G.U. Xianfeng, Dimitris Samaras, A two-step computation of the exact GAN Wasserstein distance, in: *International Conference on Machine Learning*, 2018.
- [41] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, *Adv. Neural Inf. Process. Syst.* (2017).
- [42] David K. Duvenaud, et al., Convolutional networks on graphs for learning molecular fingerprints, *Adv. Neural Inf. Process. Systems* (2015).
- [43] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint [arXiv:1609.02907](https://arxiv.org/abs/1609.02907).
- [44] Sepp Hochreiter, Jürgen Schmidhuber, Long short-term memory, *Neural Comput.* 9 (8) (1997) 1735–1780.
- [45] Richard S. Sutton, et al., Policy gradient methods for reinforcement learning with function approximation, *Adv. Neural Inf. Process. Systems* (2000).
- [46] Tim Dettmers, et al., Convolutional 2d knowledge graph embeddings, in: *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [47] Kristina Toutanova, et al., Representing text for joint embedding of text and knowledge bases, in: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 2015.
- [48] Wenhan Xiong, Thien Hoang, William Yang Wang, Deeppath: A reinforcement learning method for knowledge graph reasoning, 2017, arXiv preprint [arXiv:1707.06690](https://arxiv.org/abs/1707.06690).
- [49] Diederik P. Kingma, Jimmy Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
- [50] Yankai Lin, et al., Learning entity and relation embeddings for knowledge graph completion, in: *Twenty-ninth AAAI conference on artificial intelligence*, 2015.
- [51] Tingsong Jiang, et al., Encoding temporal information for time-aware link prediction, in: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016.
- [52] Luis Galárraga, et al., Fast rule mining in ontological knowledge bases with AMIE \$\$\$\$, *VLDB J.* 24 (6) (2015) 707–730.

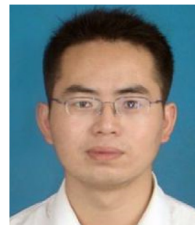


Qi Wang received the B.S. and M.Eng. degree in Software engineering from Jilin University (Changchun city) and Central South University (Changsha city) in 2012 and 2016, China, respectively. He is currently pursuing the Ph.D. degree in Software engineering at the school of computer science, Fudan University, Shanghai, China.

His current research interests include knowledge graph, deep learning, and reinforcement learning.



Yuede Ji is a Ph.D. student from the Department of Electrical and Computer Engineering at The George Washington University advised by **Prof. H. Howie Huang**. His research interests are mainly in graph analytics for high performance computing, cybersecurity, and machine learning. Prior to joining GWU, He received B.E. and M.S. both from Jilin University.



Yongsheng Hao received his MS Degree of Engineering from Qingdao University in 2008. Now, he is a senior engineer of Network Center, Nanjing University of Information Science & Technology. His current research interests include distributed and parallel computing, mobile computing, Grid computing, web Service, particle swarm optimization algorithm and genetic algorithm. He has published more than 30 papers in international conferences and journals.



Jie Cao received the Ph.D. degree from Southeast University, Nanjing, China, in 2005. He was an Associate Professor, from 1999 to 2006. From 2006 to 2009, he was a Postdoctoral Fellow of the Academy of Mathematics and Systems Science, Chinese Academy of Science. From 2009 to 2019, he was a Professor with the School of Management and Economics, Nanjing University of Information Science and Technology. Since May 2019, he has been the Vice President of the Xuzhou University of Technology. His research interests include system engineering, and management science

and technology.